Pontificia Universidad Javeriana Cali

Facultad de Ingeniería y Ciencias

Actividad guiada: Machine Learning supervisado y no supervisado

Clasificación de pacientes con diabetes y segmentación de clientes con datos sintéticos

Brayan David Zuluaga

Sistemas Inteligentes

Santiago de Cali, octubre de 2026

Índice

1. Introducción **2**

2. Parte 1: ML supervisado – predicción de diabetes **2**
2.1. Descripción del problema . . . 2
2.2. Distribuciones seleccionadas . . . 2
2.3. Código utilizado para generar los datos . . . 3
2.4. Análisis exploratorio . . . 4
2.5. Preparación de los datos . . . 6
2.6. Algoritmos y parámetros . . . 6
2.7. Resultados . . . 6
2.8. Efecto del tamaño del conjunto de datos . . . 7
2.9. Respuestas a las preguntas . . . 8

3. Parte 2: ML no supervisado – segmentación de clientes **9**
3.1. Descripción del problema . . . 9
3.2. Distribuciones seleccionadas . . . 9
3.3. Código utilizado para generar los datos . . . 9
3.4. Análisis exploratorio . . . 10
3.5. Algoritmos y parámetros . . . 11
3.6. Selección del número de grupos . . . 12
3.7. Comparación de los algoritmos . . . 13
3.8. Interpretación de los clusters . . . 13
3.9. Respuestas a las preguntas . . . 14

4. Conclusiones **15**

1. Introducción

En esta actividad trabajé con los dos enfoques clásicos del aprendizaje automático. En la primera parte armé un conjunto de datos sintético de pacientes y entrené dos clasificadores para predecir si una persona tiene diabetes. En la segunda parte generé datos de clientes de un supermercado y usé tres algoritmos de agrupamiento para encontrar segmentos sin decirle al modelo a qué grupo pertenece cada cliente.

Como los datos son sintéticos, tuve control total sobre cómo se generan, lo cual es muy útil: puedo verificar si los modelos realmente descubren la estructura que yo mismo metí en los datos. Todo se hizo en Python con NumPy, Pandas, Matplotlib, SciPy y scikit-learn, usando siempre la semilla `42` para que los resultados se puedan reproducir.

2. Parte 1: ML supervisado – predicción de diabetes

2.1. Descripción del problema

La idea es simular la información médica de 2.000 pacientes con cinco variables predictoras: edad, nivel de glucosa en sangre, índice de masa corporal (IMC), presión arterial y antecedentes familiares. Cada paciente tiene una etiqueta que dice si tiene diabetes (*Sí*) o no (*No*). Con eso se entrenan dos algoritmos de clasificación y se compara cuál funciona mejor.

2.2. Distribuciones seleccionadas

Escogí cada distribución pensando en que tuviera sentido para la variable que representa:

Tabla 1: Distribuciones usadas para las variables predictoras

Variable	Distribución	Parámetros	Por qué
Edad (años)	Normal truncada	$\mu = 50,\ \sigma = 15$, en $[18, 90]$	Población adulta, sin edades imposibles
Glucosa (mg/dL)	Normal	$\mu = 100,\ \sigma = 20$	Variable fisiológica simétrica
IMC (kg/m^2)	Lognormal	mediana $= 27$, $\sigma_{\log} = 0{,}18$	Siempre positiva y con cola a la derecha
Presión arterial (mmHg)	Normal	$\mu = 125,\ \sigma = 15$	Variable fisiológica simétrica
Antecedentes familiares	Bernoulli	$p = 0{,}30$	Es una variable de sí/no

Regla para construir la etiqueta. Como los datos son inventados, la etiqueta de diabetes tiene que salir de una fórmula conocida. Usé un puntaje lineal con ruido que luego paso por una sigmoide:

$$\begin{aligned} z = {} & -11{,}5 + 0{,}03\,\text{edad} + 0{,}05\,\text{glucosa} + 0{,}08\,\text{IMC} \\ & + 0{,}015\,\text{presión} + 1{,}0\,\text{antecedentes} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0;\, 0{,}75) \end{aligned} \tag{1}$$

$$p = \frac{1}{1 + e^{-z}}, \qquad \text{diabetes} = \begin{cases} \text{Sí} & \text{si } p \geq 0{,}5 \ (z \geq 0) \\ \text{No} & \text{en otro caso} \end{cases} \tag{2}$$

El término de ruido ε representa factores que no medimos (dieta, actividad física, genética, etc.). Gracias a ese ruido las clases no quedan perfectamente separadas, que es lo que pasaría con datos reales.

2.3. Código utilizado para generar los datos

```python
# Parámetros teóricos de las distribuciones (se usan para generar Y para las pruebas)
PARAM = {
    "edad": {"dist": "Normal truncada", "mu": 50.0, "sigma": 15.0, "a": 18.0, "b": 90.0},
    "glucosa": {"dist": "Normal", "mu": 100.0, "sigma": 20.0},
    "imc": {"dist": "LogNormal", "mu_log": float(np.log(27.0)), "sigma_log": 0.18},
    "presion": {"dist": "Normal", "mu": 125.0, "sigma": 15.0},
    "antecedentes": {"dist": "Bernoulli", "p": 0.30},
}

# Coeficientes de la regla de etiquetado (score logístico)
REGLA = {
    "intercepto": -11.5,
    "coef": {"edad": 0.03, "glucosa": 0.05, "imc": 0.08, "presion": 0.015, "antecedentes": 1.0},
    "sigma_ruido": 0.75,
    "umbral_probabilidad": 0.5,
}

FEATURES = ["edad", "glucosa", "imc", "presion", "antecedentes"]
VARS_CONTINUAS = ["edad", "glucosa", "imc", "presion"]


def dist_edad():
    """Distribución Normal(50,15) truncada a [18, 90] (scipy usa límites estandarizados)."""
    p = PARAM["edad"]
    a, b = (p["a"] - p["mu"]) / p["sigma"], (p["b"] - p["mu"]) / p["sigma"]
    return stats.truncnorm(a, b, loc=p["mu"], scale=p["sigma"])


def dist_imc():
    """LogNormal: si X ~ LogNormal(mu, s) entonces ln X ~ Normal(mu, s)."""
    p = PARAM["imc"]
    return stats.lognorm(s=p["sigma_log"], scale=np.exp(p["mu_log"]))


DIST_TEORICA = {
    "edad": dist_edad(),
    "glucosa": stats.norm(PARAM["glucosa"]["mu"], PARAM["glucosa"]["sigma"]),
    "imc": dist_imc(),
    "presion": stats.norm(PARAM["presion"]["mu"], PARAM["presion"]["sigma"]),
}


# ---------------------------------------------------------------------------
# 1. Generación de datos sintéticos
# ---------------------------------------------------------------------------
def score_lineal(df):
    """Combinación lineal (sin ruido) de la regla de etiquetado."""
    z = REGLA["intercepto"]
    for var, c in REGLA["coef"].items():
        z = z + c * df[var]
    return z


def generar_datos(n, seed=SEED):
    """Genera n pacientes sintéticos y su etiqueta según la regla documentada arriba."""
    rng = np.random.default_rng(seed)
    df = pd.DataFrame({
        "edad": DIST_TEORICA["edad"].rvs(size=n, random_state=rng),
        "glucosa": rng.normal(PARAM["glucosa"]["mu"], PARAM["glucosa"]["sigma"], n),
        "imc": rng.lognormal(PARAM["imc"]["mu_log"], PARAM["imc"]["sigma_log"], n),
        "presion": rng.normal(PARAM["presion"]["mu"], PARAM["presion"]["sigma"], n),
        "antecedentes": rng.binomial(1, PARAM["antecedentes"]["p"], n),
    })
    ruido = rng.normal(0.0, REGLA["sigma_ruido"], n)
    z = score_lineal(df) + ruido
    prob = 1.0 / (1.0 + np.exp(-z))                    # sigmoide
```

```
    df["diabetes"] = np.where(prob >= REGLA["umbral_probabilidad"], "Sí", "No")
    return df


df = generar_datos(N_PRINCIPAL, SEED)
df_csv = df.copy()
for c in ["edad", "glucosa", "imc", "presion"]:
    df_csv[c] = df_csv[c].round(2)
df_csv.to_csv(os.path.join(BASE, "datos_diabetes.csv"), index=False, encoding="utf-8-sig")
print(f"[1] Datos generados: {df.shape[0]} registros x {df.shape[1]} columnas -> datos_diabetes.csv")
```

El código completo (análisis exploratorio, modelos y métricas) está en `supervisado/supervisado.py`.

2.4. Análisis exploratorio

Tabla 2: Estadísticas descriptivas ($n = 2.000$)

Variable	Media	Desv.	Mín.	Q1	Mediana	Q3	Máx.	Asim.
Edad	50,66	14,15	18,18	40,14	50,09	60,70	89,30	0,15
Glucosa	100,40	19,90	38,72	86,73	100,33	113,57	161,77	0,06
IMC	27,50	5,02	12,25	23,93	27,10	30,35	50,28	0,58
Presión	124,75	15,00	79,06	115,17	124,09	135,02	174,41	0,02
Antecedentes	0,29	0,46	0	0	0	1	1	0,90

Las medias y desviaciones quedaron muy cerca de los parámetros que puse. El IMC es la única continua con asimetría positiva clara (0,58), lo cual es justo lo que se espera de una lognormal.

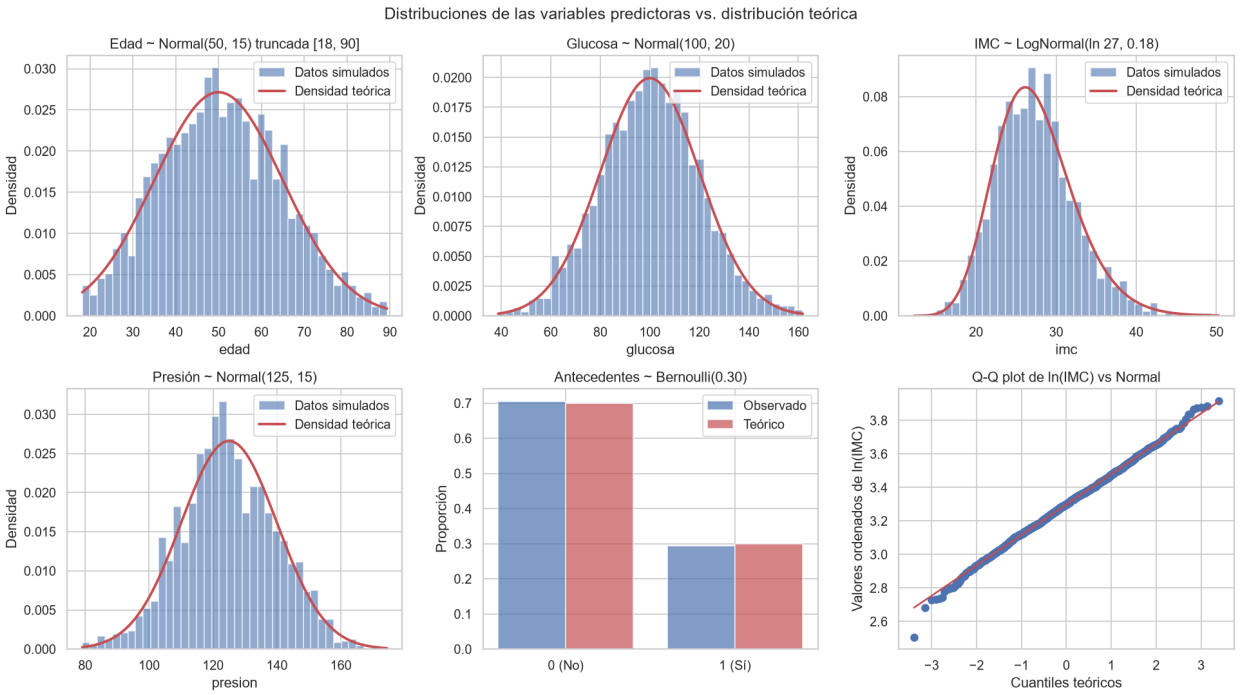


Figura 1: Histogramas de cada variable con la densidad teórica superpuesta

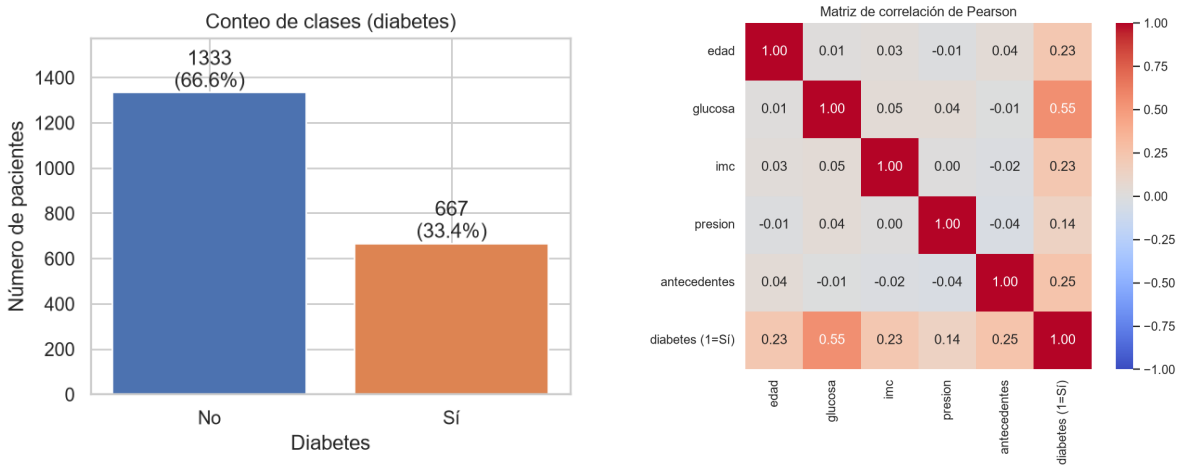


Figura 2: Conteo de clases (izq.) y matriz de correlación (der.)

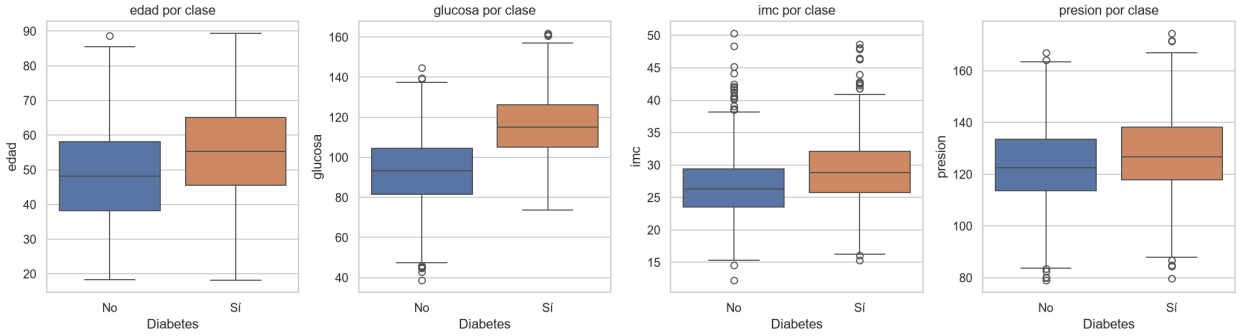


Figura 3: Distribución de las variables según la clase

En los diagramas de caja se ve que la glucosa es la variable que más separa a los diabéticos de los que no lo son, seguida del IMC y la edad. Esto cuadra con la regla, porque la glucosa es la que más pesa en z cuando se tiene en cuenta su escala.

Pruebas de bondad de ajuste. Para comprobar formalmente las distribuciones usé Kolmogorov-Smirnov contra la distribución teórica exacta, Shapiro-Wilk para normalidad y chi-cuadrado para la variable binaria.

Tabla 3: Pruebas de bondad de ajuste ($\alpha = 0{,}05$; Bonferroni $\alpha = 0{,}0056$)

Variable	**Prueba**	**Estadístico**	**p-valor**	$\alpha = 0{,}05$	**Bonferroni**
Edad	KS vs. normal truncada	0,0176	0,557	No rechaza	No rechaza
Glucosa	KS vs. normal	0,0179	0,540	No rechaza	No rechaza
IMC	KS vs. lognormal	0,0177	0,552	No rechaza	No rechaza
Presión	KS vs. normal	0,0363	0,010	Rechaza	No rechaza
Glucosa	Shapiro-Wilk	0,9990	0,361	No rechaza	No rechaza
Presión	Shapiro-Wilk	0,9983	0,039	Rechaza	No rechaza
ln(IMC)	Shapiro-Wilk	0,9987	0,124	No rechaza	No rechaza
Edad	Shapiro-Wilk (sin truncar)	0,9943	$< 0{,}001$	Rechaza	Rechaza
Antecedentes	Chi-cuadrado	0,2881	0,591	No rechaza	No rechaza

2.5. Preparación de los datos

- Separé las cinco variables predictoras en X y la etiqueta en y (Sí = 1, No = 0).
- Dividí en 80 % entrenamiento (1.600 registros) y 20 % prueba (400) usando `train_test_split` con `stratify=y` y `random_state=42`. Al ser estratificada, las dos partes quedaron con el mismo 33,3 % de casos positivos.
- Para la regresión logística estandaricé las variables con `StandardScaler`, ajustándolo solo con los datos de entrenamiento para no filtrar información del conjunto de prueba.

2.6. Algoritmos y parámetros

Regresión logística. Es un modelo lineal que estima la probabilidad de que el paciente tenga diabetes aplicando la sigmoide a una combinación lineal de las variables. Lo usé dentro de un `Pipeline` con `StandardScaler`, con `C=1.0` (regularización L2), *solver* `lbfgs` y `max_iter=1000`.

Random Forest. Es un conjunto de árboles de decisión, cada uno entrenado con una muestra *bootstrap* de los datos y un subconjunto aleatorio de variables; la predicción final sale por votación. Usé `n_estimators=300`, `max_depth=None`, `min_samples_leaf=2`, `max_features='sqrt'` y `bootstrap=True`. No necesita escalar las variables.

2.7. Resultados

Tabla 4: Métricas sobre el conjunto de prueba (400 pacientes)

Modelo	Accuracy	Precision	Recall	F1-score	Acc. entrenamiento
Regresión logística	**0,855**	0,774	**0,797**	**0,785**	0,858
Random Forest	0,845	**0,775**	0,752	0,763	0,978
Regla real sin ruido	0,845	0,752	0,797	0,774	–

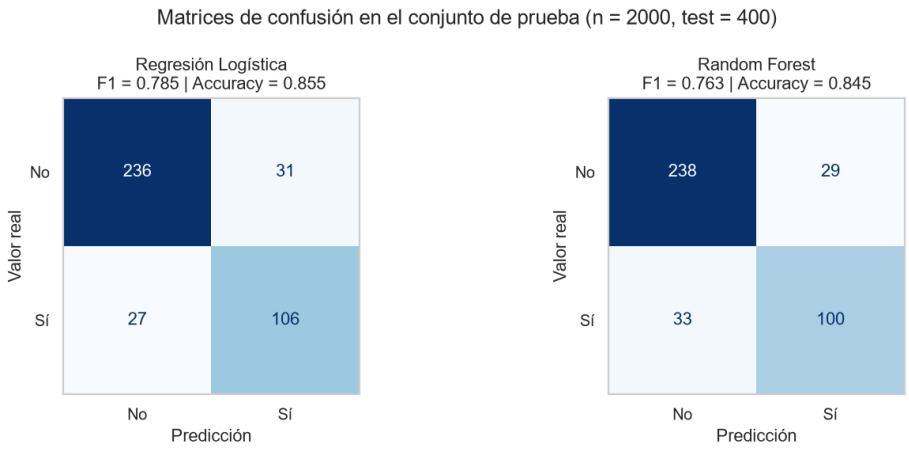


Figura 4: Matrices de confusión en el conjunto de prueba

Tabla 5: Matrices de confusión (filas: real; columnas: predicho)

Modelo	VN	FP	FN	VP
Regresión logística	236	31	27	106
Random Forest	238	29	33	100

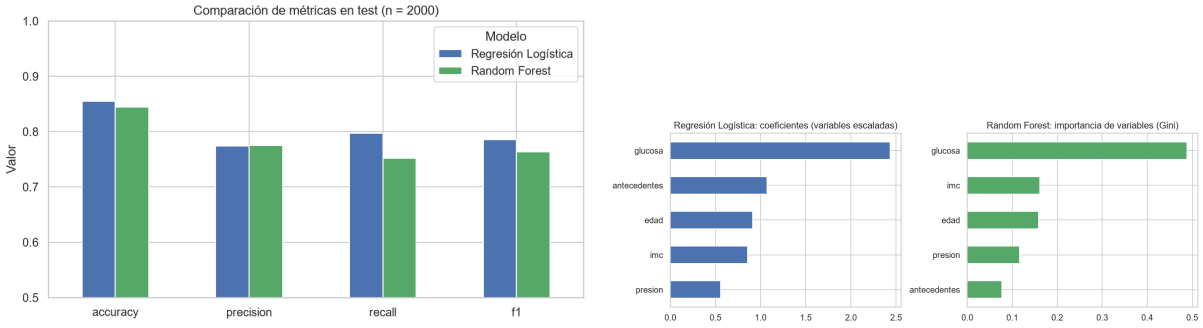


Figura 5: Comparación de métricas (izq.) e importancia de variables en Random Forest (der.)

La importancia de variables del Random Forest confirma lo que se veía en el análisis exploratorio: la glucosa pesa casi la mitad (0,49), y luego vienen IMC (0,16), edad (0,16), presión (0,12) y antecedentes (0,08).

2.8. Efecto del tamaño del conjunto de datos

Repetí todo el proceso con 500, 2.000 y 10.000 registros, cinco veces por tamaño con semillas distintas, y promedié las métricas.

Tabla 6: Métricas promedio según el número de registros (media de 5 repeticiones)

n	Modelo	Accuracy	Precision	Recall	F1 (± desv.)	Acc. entren.
500	Regresión logística	0,834	0,764	0,715	0,737 ± 0,047	0,861
500	Random Forest	0,802	0,712	0,667	0,685 ± 0,058	0,983
2.000	Regresión logística	0,853	0,801	0,746	0,771 ± 0,023	0,849
2.000	Random Forest	0,837	0,774	0,721	0,745 ± 0,020	0,976
10.000	Regresión logística	0,845	0,792	0,729	0,759 ± 0,009	0,851
10.000	Random Forest	0,834	0,770	0,718	0,743 ± 0,011	0,978

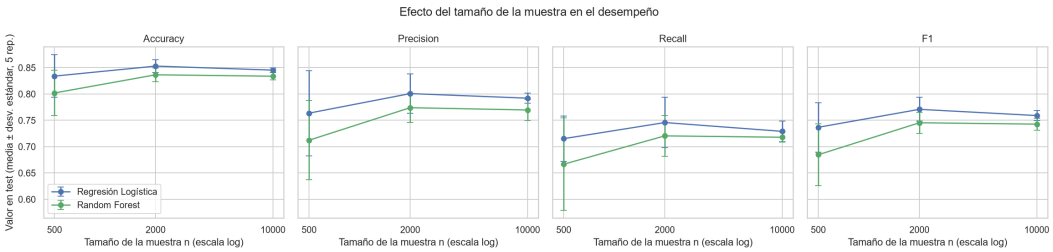


Figura 6: Métricas en función del número de registros

2.9. Respuestas a las preguntas

1. ¿Las variables generadas siguen las distribuciones estadísticas esperadas? Sí. Para edad, glucosa, IMC y antecedentes ninguna prueba rechaza la hipótesis de que siguen la distribución teórica, y los histogramas coinciden muy bien con las curvas. La presión arterial dio $p = 0{,}010$ en KS y 0,039 en Shapiro, lo que a primera vista parecería un rechazo. Pero como hice nueve pruebas a la vez, lo correcto es aplicar la corrección de Bonferroni ($\alpha = 0{,}0056$), y con eso ya no se rechaza. Además, la presión se generó exactamente con una normal, así que ese resultado es un error tipo I, algo que pasa aproximadamente una de cada veinte veces por puro azar. La edad no pasa la prueba de normalidad *sin truncar*, pero eso era lo esperado porque la corté entre 18 y 90 años; contra la normal truncada sí se ajusta.

2. ¿Qué proporción de pacientes pertenece a cada clase? 1.333 pacientes sin diabetes (66,65 %) y 667 con diabetes (33,35 %). Es un desbalance moderado de más o menos 2 a 1, y por eso fue importante usar la división estratificada.

3. ¿Cuál de los algoritmos obtiene el mejor F1-score? La regresión logística, con F1 = 0,785 frente a 0,763 del Random Forest. También gana en accuracy y recall; en precisión quedan prácticamente empatados. Su F1 queda incluso a la par de la regla real sin ruido (0,774), o sea que llegó prácticamente al techo que permiten los datos.

4. ¿Qué sucede cuando se cambia el número de registros de 500 a 2.000 o 10.000? De 500 a 2.000 los dos modelos mejoran, sobre todo el Random Forest (su F1 sube de 0,685 a 0,745), porque es un modelo más flexible que necesita más ejemplos. De 2.000 a 10.000 las métricas prácticamente no cambian: el límite ya no lo pone la cantidad de datos, sino el ruido de la etiqueta. Lo que sí mejora bastante es la estabilidad: la desviación del F1 baja de cerca de 0,05 a cerca de 0,01, es decir, con más datos los resultados dependen mucho menos de qué pacientes caigan en la prueba. También se ve que el Random Forest sobreajusta en todos los tamaños (alrededor de 0,98 en entrenamiento contra 0,83 en prueba).

5. ¿Qué ventajas y desventajas presenta cada algoritmo? En la Tabla 7 resumo lo que observé de cada uno en esta actividad.

Tabla 7: Ventajas y desventajas de cada algoritmo

	Ventajas	**Desventajas**
Regresión logística	Es simple, rápida e interpretable: los coeficientes dicen cuánto pesa cada variable. Entrega probabilidades bien calibradas, casi no sobreajusta y funciona bien con pocos datos.	Asume que la frontera de decisión es lineal, así que no capta interacciones ni relaciones no lineales a menos que uno las construya a mano. Requiere escalar las variables.
Random Forest	Capta relaciones no lineales e interacciones sin tener que especificarlas, no necesita escalado, es robusto a datos atípicos y da la importancia de cada variable.	Es más pesado de entrenar y mucho menos interpretable. Tiende a memorizar el ruido (sobreajuste) y necesita más datos para rendir bien.

3. Parte 2: ML no supervisado – segmentación de clientes

3.1. Descripción del problema

Aquí el objetivo es distinto: no hay una etiqueta que predecir. Generé 3.200 clientes de un supermercado con su edad, ingresos anuales, frecuencia de compra mensual, valor promedio del tiquete y gasto mensual, y la idea es que los algoritmos encuentren por sí solos grupos de clientes parecidos. Usé K-means, K-medoids y clustering jerárquico.

3.2. Distribuciones seleccionadas

Para que existieran grupos reales que encontrar, generé los datos como una mezcla de cuatro segmentos. La etiqueta del segmento **no** se le pasa a ningún algoritmo; solo la guardé para comprobar al final qué tan bien se recuperaron los grupos (con el índice ARI).

Tabla 8: Parámetros de los segmentos latentes

Segmento	n	Edad	Ingresos (lognormal)	Visitas (Poisson)	Tiquete (gamma)
Jóvenes/estudiantes	800	Uniforme(18, 28)	med. 14.000, $\sigma = 0{,}35$	$\lambda = 5$	(4; 5)
Familias	1.000	Normal(42, 6)	med. 38.000, $\sigma = 0{,}30$	$\lambda = 14$	(6; 9)
Profesionales premium	600	Normal(33, 4)	med. 90.000, $\sigma = 0{,}25$	$\lambda = 3$	(6; 18)
Adultos mayores	800	Normal(68, 5)	med. 22.000, $\sigma = 0{,}30$	$\lambda = 16$	(5; 3,5)

Los ingresos son lognormales porque el dinero siempre es positivo y tiene cola larga a la derecha; la frecuencia es Poisson porque es un conteo de visitas; y el tiquete es gamma por ser positivo y asimétrico. El tiquete se multiplica por (ingresos/mediana)^{0,3} para que la gente con más plata gaste un poco más por visita, y el gasto mensual es frecuencia $\times$ tiquete $\times\, \mathcal{N}(1; 0{,}05)$.

3.3. Código utilizado para generar los datos

```

# ------------------------------------------------------------------------
# 1. GENERACIÓN DE DATOS
# ------------------------------------------------------------------------
# Se simulan 4 segmentos latentes (mezcla). La etiqueta latente NO se usa para
# agrupar; sólo se guarda para validar a posteriori (ARI).
#
# Variables:
#  - edad               : Normal(mu, sigma) o Uniforme(a, b) según segmento
#  - ingresos_anuales   : LogNormal(ln(mediana), sigma)      [USD/año]
#  - frecuencia_mensual : Poisson(lambda)                    [visitas/mes]
#  - ticket_promedio    : Gamma(forma, escala) * f(ingresos) [USD/visita]
#  - gasto_mensual      : frecuencia * ticket (+ ruido)      [USD/mes]
SEGMENTOS = {
    "Jóvenes/estudiantes": dict(n=800, edad=("uniforme", 18, 28),
                                ingreso_mediana=14000, ingreso_sigma=0.35,
                                lam=5, gamma_forma=4.0, gamma_escala=5.0),
    "Familias": dict(n=1000, edad=("normal", 42, 6),
                     ingreso_mediana=38000, ingreso_sigma=0.30,
                     lam=14, gamma_forma=6.0, gamma_escala=9.0),
    "Profesionales premium": dict(n=600, edad=("normal", 33, 4),
                                  ingreso_mediana=90000, ingreso_sigma=0.25,
                                  lam=3, gamma_forma=6.0, gamma_escala=18.0),
    "Adultos mayores": dict(n=800, edad=("normal", 68, 5),
                            ingreso_mediana=22000, ingreso_sigma=0.30,
                            lam=16, gamma_forma=5.0, gamma_escala=3.5),
}
```

```python

partes = []
for seg_id, (nombre, p) in enumerate(SEGMENTOS.items()):
    n = p["n"]
    tipo, a, b = p["edad"]
    edad = rng.uniform(a, b, n) if tipo == "uniforme" else rng.normal(a, b, n)
    edad = np.clip(np.round(edad), 18, 90)
    ingresos = rng.lognormal(np.log(p["ingreso_mediana"]), p["ingreso_sigma"], n)
    frecuencia = np.maximum(rng.poisson(p["lam"], n), 1)
    # ticket: gamma, levemente modulado por los ingresos del cliente
    factor = (ingresos / p["ingreso_mediana"]) ** 0.3
    ticket = rng.gamma(p["gamma_forma"], p["gamma_escala"], n) * factor
    gasto = frecuencia * ticket * rng.normal(1.0, 0.05, n)
    partes.append(pd.DataFrame({
        "edad": edad, "ingresos_anuales": ingresos.round(2),
        "frecuencia_mensual": frecuencia, "ticket_promedio": ticket.round(2),
        "gasto_mensual": np.maximum(gasto, 1).round(2),
        "segmento_latente": seg_id}))

df = pd.concat(partes, ignore_index=True).sample(frac=1, random_state=SEED)
df = df.reset_index(drop=True)
df.insert(0, "id_cliente", np.arange(1, len(df) + 1))
df.to_csv(os.path.join(BASE, "datos_clientes.csv"), index=False)
```

3.4. Análisis exploratorio

Tabla 9: Estadísticas descriptivas ($n = 3.200$)

Variable	Media	Desv.	Mín.	Mediana	Máx.	Asim.	Curt.
Edad (años)	41,9	17,2	18	38	83	0,55	−0,96
Ingresos (USD/año)	39.579	31.204	3.904	28.158	172.657	1,51	1,66
Frecuencia (visitas/mes)	10,1	6,3	1	10	31	0,27	−0,95
Tiquete (USD)	47,5	42,8	2,1	31,6	374,2	1,76	3,79
Gasto (USD/mes)	393,2	361,0	4,5	284,9	3.112,3	1,78	4,55

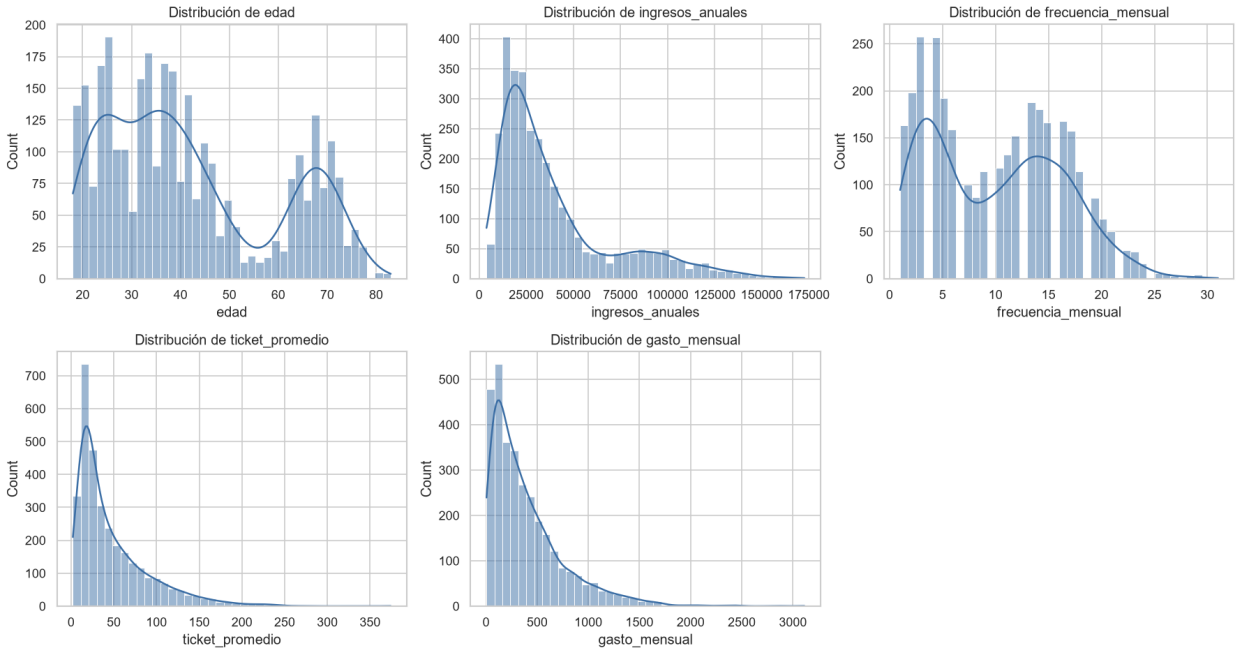


Figura 7: Histogramas de las variables de clientes

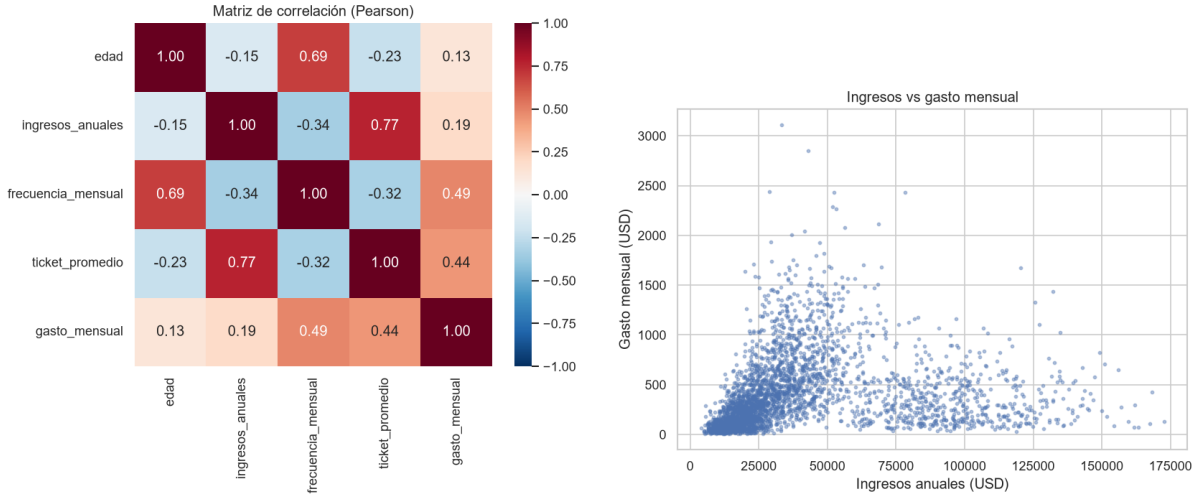


Figura 8: Matriz de correlación (izq.) y relación ingresos–gasto (der.)

Ingresos, tiquete y gasto tienen una asimetría fuerte (entre 1,5 y 1,8), así que antes de agrupar les apliqué $\log(1+x)$ y luego estandaricé todo con `StandardScaler`. Si no se hace esto, los algoritmos basados en distancias quedan dominados por las variables de dinero, que tienen escalas mucho más grandes. Las correlaciones más fuertes fueron ingresos–tiquete (0,77) y edad–frecuencia (0,69).

3.5. Algoritmos y parámetros

- **K-means:** `KMeans(n_clusters=k, n_init=10, random_state=42)`. Asigna cada cliente al centroide más cercano y minimiza la inercia (suma de distancias al cuadrado).

- **K-medoids:** lo implementé directamente en NumPy porque la librería `scikit-learn-extra` no se pudo instalar en la versión de Python que estaba usando. Usa una inicialización tipo *k-medoids++*, alterna entre asignar clientes y actualizar medoides, y hace 5 reinicios. La diferencia con K-means es que los centros son clientes reales, lo que lo hace más robusto a atípicos.
- **Jerárquico aglomerativo:** `AgglomerativeClustering(n_clusters=k, linkage='ward')`. Empieza con cada cliente como un grupo y los va uniendo minimizando el aumento de varianza. El dendrograma lo dibujé con una muestra de 300 clientes para que se pudiera leer.

3.6. Selección del número de grupos

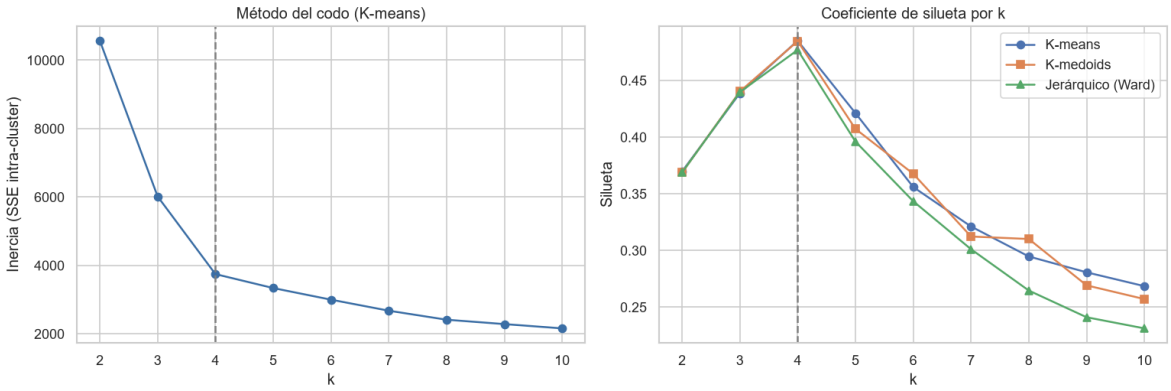


Figura 9: Método del codo (inercia) y coeficiente de silueta para $k = 2, \ldots, 10$

La inercia baja fuerte hasta $k = 4$ (10.569 → 6.016 → 3.745) y de ahí en adelante baja poquito (3.337 en $k = 5$), así que el codo está en 4. La silueta también es máxima en $k = 4$ para los tres algoritmos (0,485 en K-means). Como los dos criterios coinciden, escogí $k = 4$, que además es el número real de segmentos con los que generé los datos.

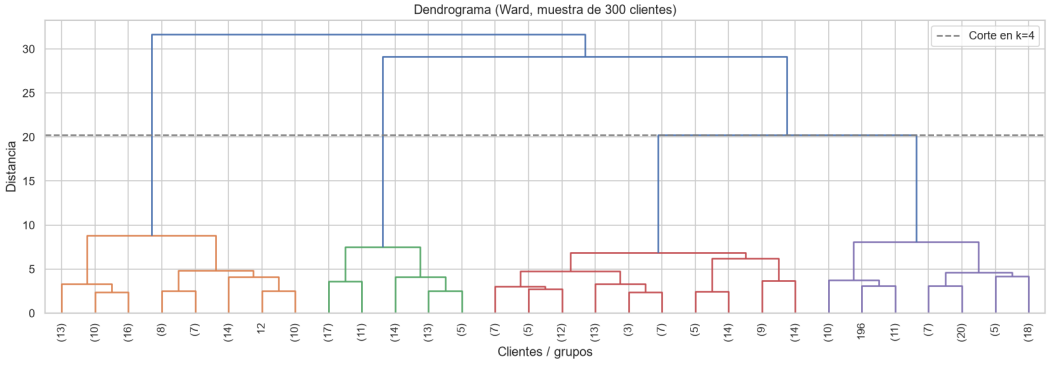


Figura 10: Dendrograma del clustering jerárquico (Ward, muestra de 300 clientes)

3.7. Comparación de los algoritmos

Tabla 10: Métricas de calidad con $k = 4$

Algoritmo	Silueta ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	ARI vs. real ↑
K-means	**0,485**	0,766	**3.486**	0,946
K-medoids	0,485	**0,762**	3.474	0,940
Jerárquico (Ward)	0,477	0,782	3.359	**0,964**

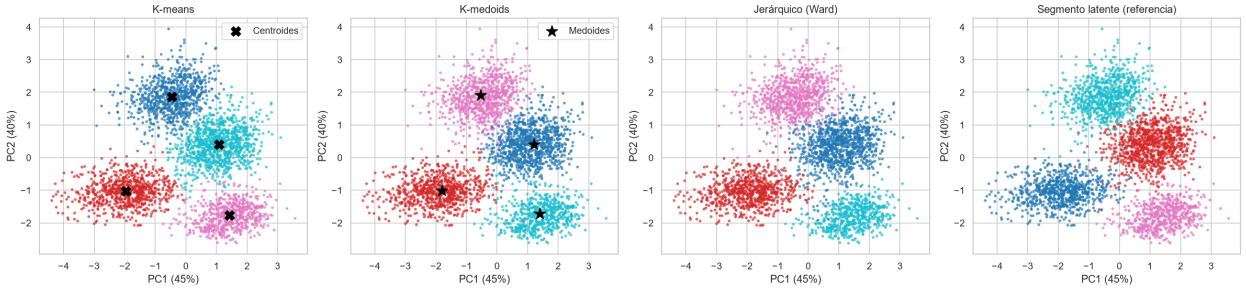


Figura 11: Clusters de cada algoritmo proyectados en 2D con PCA (85 % de la varianza)

3.8. Interpretación de los clusters

Tabla 11: Promedios por cluster (K-means)

C.	Perfil	Edad	Ingresos	Visitas	Tiquete	Gasto	%
0	Adultos mayores	67,5	23.037	16,0	17,6	282	25,4
1	Jóvenes/estudiantes	23,2	14.738	4,8	19,5	93	24,8
2	Profesionales premium	33,2	94.361	3,1	112,1	343	19,1
3	Familias	41,4	39.281	13,8	54,8	758	30,8

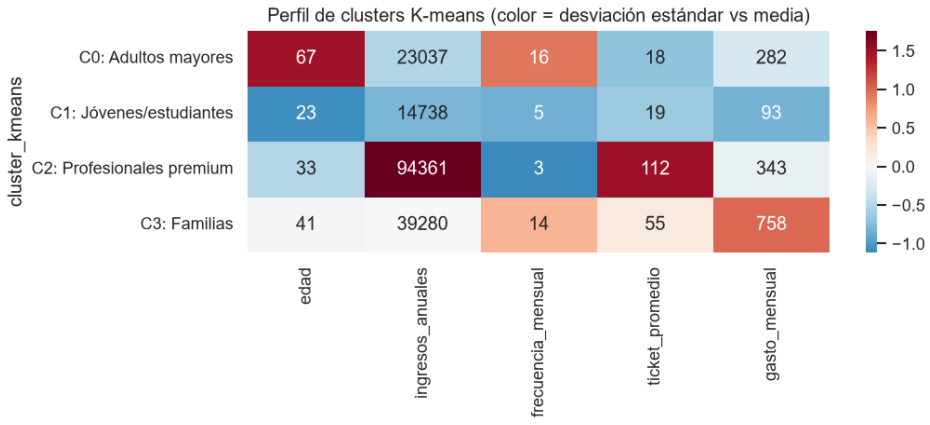


Figura 12: Perfil promedio estandarizado de cada cluster

- **Adultos mayores (25 %):** van al supermercado casi todos los días (16 veces al mes), pero compran poquito en cada visita. Ingresos medios-bajos.
- **Jóvenes/estudiantes (25 %):** los de menores ingresos y menor gasto; van pocas veces y con tiquetes pequeños.
- **Profesionales premium (19 %):** los de mayores ingresos. Van muy poco (3 veces al mes), pero cada compra es grande (más de 110 USD).
- **Familias (31 %):** el grupo más valioso para el supermercado, porque combinan buena frecuencia con un tiquete medio y terminan siendo los que más gastan al mes (758 USD).

3.9. Respuestas a las preguntas

1. ¿Qué resultados obtuvo? Sin usar ninguna etiqueta, los tres algoritmos encontraron prácticamente los mismos cuatro grupos, con una calidad de agrupamiento entre moderada y buena (silueta alrededor de 0,48 y Davies-Bouldin alrededor de 0,76). K-means y K-medoids quedaron casi empatados y coinciden entre sí en un 97,6 % (ARI), mientras que el jerárquico con Ward fue el que mejor recuperó los segmentos reales (ARI de 0,964). Las variables que más diferencian a los grupos son la edad, la frecuencia de visita y el valor del tiquete.

2. ¿Para qué serviría este análisis? Para segmentar clientes y tomar decisiones de mercadeo y operación:

- Hacer promociones dirigidas: descuentos por volumen para las familias, productos gourmet o domicilios para los profesionales premium, beneficios de fidelidad en horas de poca afluencia para los adultos mayores y marca propia o precios bajos para los jóvenes.
- Ajustar el surtido de productos y el personal según los horarios de cada grupo.
- Diseñar programas de lealtad diferenciados.
- Identificar segmentos con riesgo de dejar de comprar o con potencial de gastar más.
- Asignar automáticamente a un cliente nuevo al segmento que le corresponde, sin etiquetarlo a mano.

4. Conclusiones

- En la parte supervisada el mejor modelo fue la **regresión logística** (F1 = 0,785 contra 0,763). La razón principal es que la etiqueta se construyó con una regla lineal pasada por una sigmoide, que es exactamente la forma del modelo logístico. El Random Forest, al ser más flexible, terminó aprendiéndose parte del ruido: casi 0,98 de accuracy en entrenamiento pero 0,85 en prueba.
- El rendimiento de cualquier modelo queda limitado por el ruido que se le metió a la etiqueta. Ninguno puede superar por mucho a la regla verdadera sin ruido, y por eso pasar de 2.000 a 10.000 registros no sube las métricas, solo las hace más estables.
- Si la regla de diabetes hubiera tenido interacciones o relaciones no lineales (por ejemplo, que la glucosa solo importe a partir de cierta edad), muy probablemente el Random Forest le hubiera ganado a la regresión logística. La elección del modelo depende de la forma real del problema.
- En la parte no supervisada, el método del codo y la silueta coincidieron en $k = 4$, que es justo el número de segmentos con el que se generaron los datos. Los tres algoritmos llegaron a grupos casi idénticos: K-means es el más rápido, K-medoids da centros que son clientes reales y es más robusto a atípicos, y el jerárquico con Ward recuperó mejor la estructura original y además permite ver la jerarquía en el dendrograma.
- Un aprendizaje práctico fue la importancia del preprocesamiento: sin la transformación logarítmica y la estandarización de las variables de dinero, los grupos no salían tan bien definidos.